# Software Engineering for Financial Technology

FinTech 512

Duke

# Teaching Assistants

Dongjie,Ma, dongjie.ma@duke.edu , dm559
Pin-Yang, Chen, pin-yang.chen@duke.edu, pc275
Sean Guo, shun.guo@duke.edu, sg512
Xinyi Peng, xinyi.peng@duke.edu, xp39
Yu Chen, yu.chen@duke.edu, yc662

# What is Software Engineering?

Software ≠ code

Engineering ≠ hacking

# What is Software Engineering?

## IEEE Definition:

The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software; that is, the application of engineering to software.

"IEEE Standard Glossary of Software Engineering Terminology," in *IEEE Std 610.12-1990* , vol., no., pp.1-84, 31 Dec. 1990, doi: 10.1109/IEEESTD.1990.101064.

# What is Software Engineering?

## IEEE Definition:

The application of a

**systematic**
disciplined
quantifiable

approach to the development, operation and maintenance of software …

In science, if you know what you are doing, you should not be doing it.

In engineering, if you do not know what you are doing, you should not be doing it.

Hamming, Richard W.. "The Art of Doing Science and Engineering: Learning to Learn"

Repeatable, not heroic

"IEEE Standard Glossary of Software Engineering Terminology," in *IEEE Std 610.12-1990* , vol., no., pp.1-84, 31 Dec. 1990, doi: 10.1109/IEEESTD.1990.101064.

# What is Software Engineering?

## IEEE Definition:

Project Management

Software Development Lifecycles/Methodologies

The application of a

systematic
**disciplined**
quantifiable

Make decisions explicit, repeatable, and accountable

approach to the development, operation and maintenance of software …

"IEEE Standard Glossary of Software Engineering Terminology," in *IEEE Std 610.12-1990* , vol., no., pp.1-84, 31 Dec. 1990, doi: 10.1109/IEEESTD.1990.101064.

# What is Software Engineering?

## IEEE Definition:

The application of a

systematic
disciplined
**quantifiable**

Estimation, testing, metrics, evidence

approach to the development, operation and maintenance of software …



"IEEE Standard Glossary of Software Engineering Terminology," in *IEEE Std 610.12-1990* , vol., no., pp.1-84, 31 Dec. 1990, doi: 10.1109/IEEESTD.1990.101064.

# And a modern extension …

Software engineering also includes deployment, monitoring, and evolution of systems in production

includes code, configuration, data, infrastructure, and the processes that operate and evolve it

# Most Creative FinTech Innovation of the Past Twenty Years …

# Software is an Enabler in Finance

- Global Payments in milliseconds
- Automated trading at scale
- Contact-less payments
- Real-time risk assessment
- Block-chain based currencies
- Personalized financial products

**Without software engineering, modern finance does not exist.**

# What’s in the System?

Pick one app you use regularly

What are **three things that are part of the system** that are *not* code.

Think and then we’ll discuss

**Automation doesn’t create fraud it changes its scale.**

# Knight Capital: When Enablement Goes Wrong

Automation, speed, and scale — both power and peril

Duke

- Major U.S. market maker:
  ~ 17% trade volume on NYSE and NASDAQ in 2012
- Built systems to:
  - trade faster than humans
  - exploit small price differences
  - operate at massive scale
- Software *enabled* their business model

Knight

# The Incident: August 1st, 2012

- From 27 July to 31 July, software manually updated to 8 servers
  - One technician failed to update the code on one of the servers
  - No review of the deployments – Legacy code ”Powder Peg” remained
- On August 1st, 7 servers handled orders correctly.
- Orders sent to the server with legacy code caused the legacy code to execute due to a re-purposed flag – orders sent repeatedly and incorrectly
- Knight Capital had 50% of the trade volume in the first 45 minutes
- Losses accumulated rapidly: $450 million

# Why Humans Couldn’t Save It

- Software acted faster than humans could react
- No effective “kill switch”
- Feedback loop amplified losses
- By the time behavior was visible, the damage was done

# This Was Not “A Bug”

- Code executed as written
- No single typo / line of code explains the failure
- No single engineer caused the problem
- Failure emerged from the *system*

The system behaved correctly according to incomplete decisions.

# Where Enablement Became Amplification

| Enabling Property | Failure Effect |
|---|---|
| Speed | Losses accumulated instantly |
| Automation | No human intervention |
| Scale | Small error → massive impact |
| Complexity | Hard to reason about behavior |

The same properties that enable innovation also amplify mistakes.

# Software vs System

- Not just a code issue
- Involved
  - Deployment process
  - Configuration management
  - Operational safeguards
  - Organizational Assumptions

# Engineering Lessons

- Powerful systems need explicit safeguards
- Assumptions must be encoded, not implied
- Behavior under failure must be designed
- Speed without discipline is dangerous

Knight Capital didn’t fail because software was risky —
it failed because software was powerful.

# Software Engineering Fundamentals

Why Systems Can Scale

Duke

# Software engineering exists to enable:

- **Managing complexity**
  → so systems can grow without becoming unmanageable

- **Coordination & communication**
  → so teams (and organizations) can scale

- **Defect prevention**
  → so innovation doesn't destroy what already works

- **Requirements & design**
  → so systems do the *right* thing, consistently

# How Do Teams Build and Evolve Complex Systems?

- When building large systems, teams need to –
  - Plan work over time
  - Coordinate activities and people
  - Manage uncertainty and change
  - Reduce risk as system evolve

These approaches form development lifecycles

Lifecycles exist to help teams build complex systems effectively, not as belief systems or ideologies

# Two Fundamental Lifecycle Tradeoffs

**Emphasize Predictability**

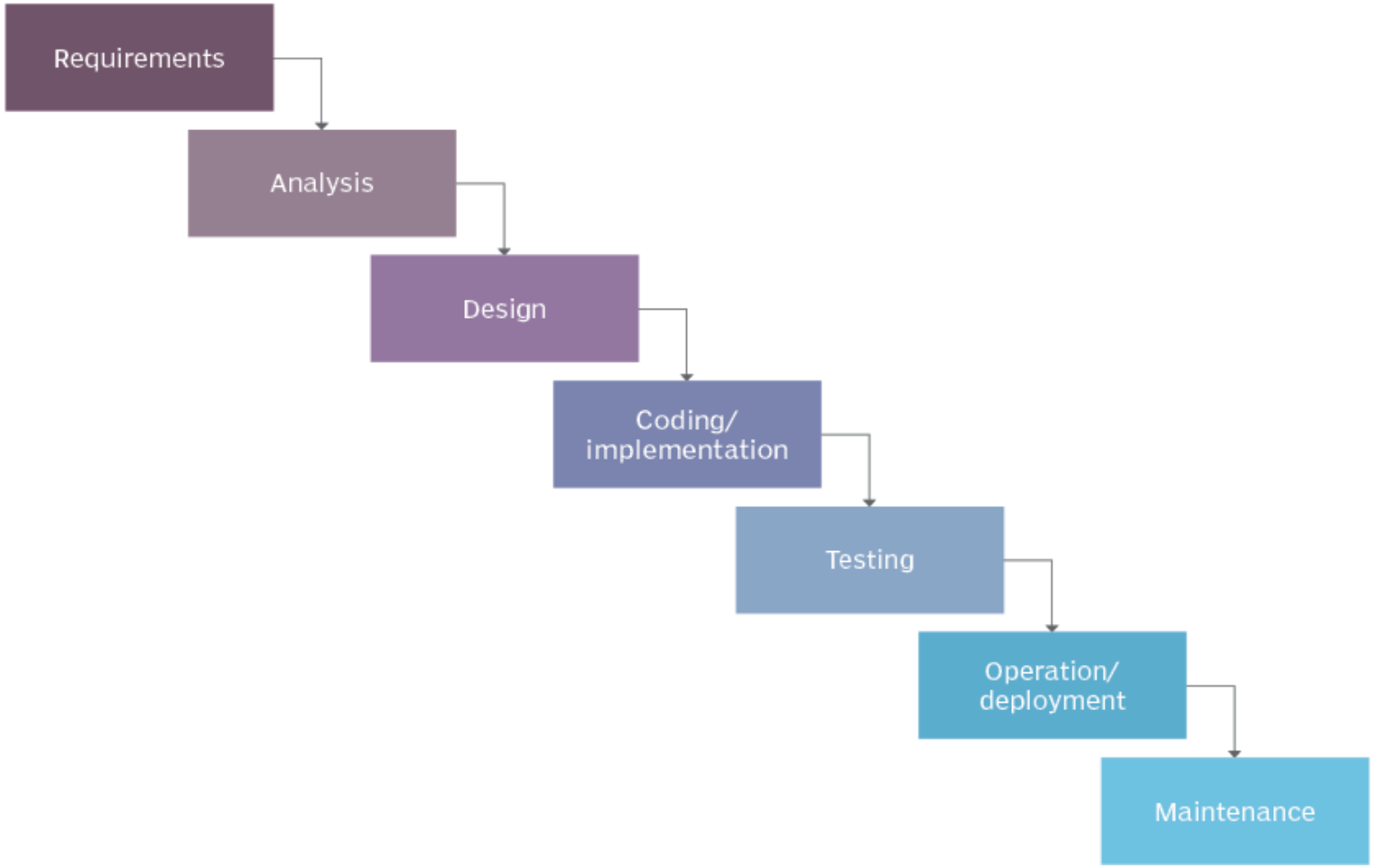
- Make decisions early
- Reduce uncertainty up front
- Favor stability and control

**Emphasize Learning**

- Make decisions incrementally
- Expect requirements to change
- Favor feedback and adaptation

Most real systems mix both approaches

# Waterfall model




https://www.educba.com/waterfall-model/

# Agile Designing for Learning and Change

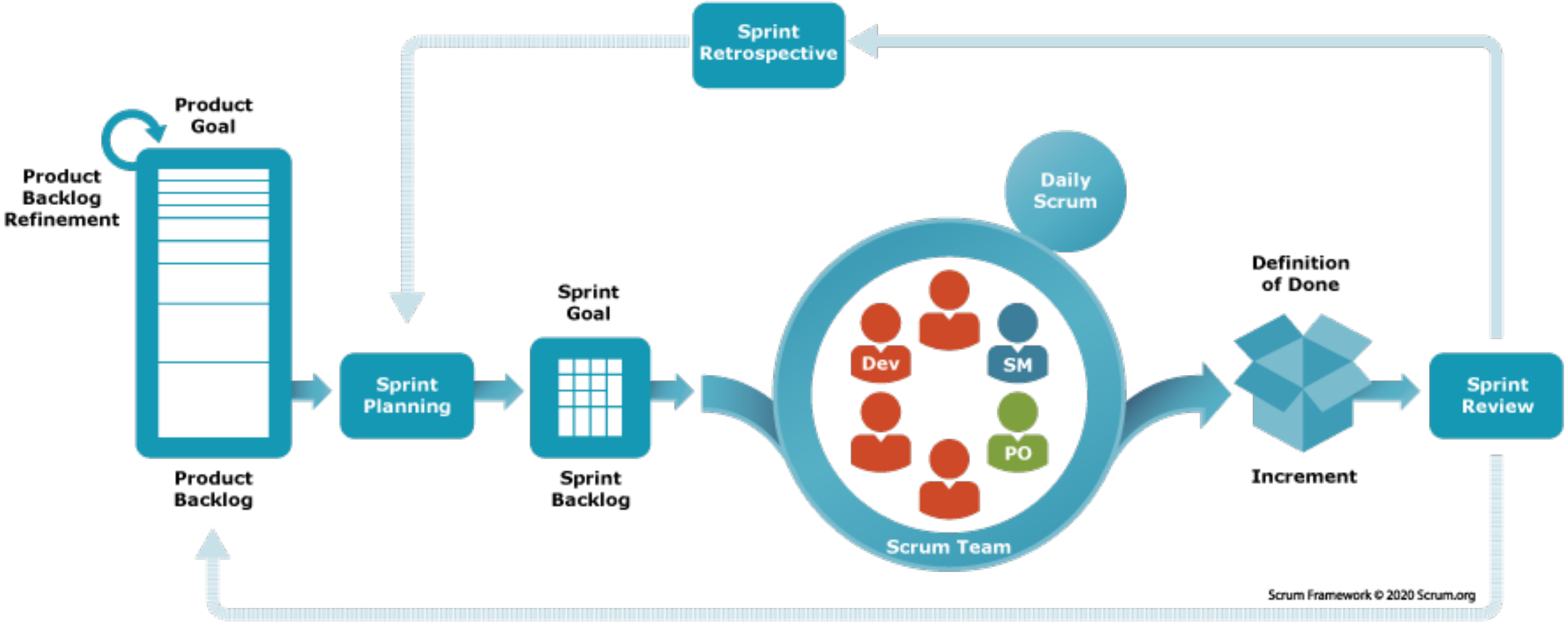
**Agile starts from a different assumption:**

- Requirements will change
- We don’t know everything up front
- Feedback improves decisions
- Working software reveals reality

**So Agile emphasizes:**

- Small, incremental changes
- Frequent feedback
- Continuous testing and integration
- Adapting plans as we learn

# Scrum




https://www.scrum.org/resources/what-scrum-module

# Project Management

## PMBOK Process Groups

### Initiating

Define a new project or a new phase of an existing project.

- Appoint project manager.
- Assess and define the scope and goals of the project.

### Planning

Establish project scope and objectives, and define how to meet objectives.

- Develop a workflow to ensure that teams meet vital benchmarks for project completion.
- Establish scope at a more granular level.
- Assess checkpoints.
- Measure and allocate project materials.
- Consider timetables.

### Executing

Complete defined work.

- Manage teams.
- Monitor timelines.
- Ensure project stays within budget and timeframe.
- Balance stakeholder involvement.
- Ensure that each step of the project is leading toward the stated outcomes from the initiating phase.

### Monitoring and Controlling

Track, review, and regulate the progress.

- Track progress and address any challenges that arise.
- Measure project execution against project planning and project outcomes.
- Review budget and reassess costs.
- Ensure dependent processes are on schedule.

### Closing

Formally close the project.

- Determine if project was completed within budget and on time.
- Perform due diligence for necessary processes.
- Assess the strengths and weaknesses of the project.
- Identify key teammates and hand out awards.

Duke

https://www.lucidchart.com/blog/what-is-pmbok

# Modern Software Engineering

- Building capabilities
- Sustaining systems over time
- Integrating people, processes, and tools
- Enabling trust in complex systems

Writing code creates features

Software engineering creates systems that can be trusted

# The Big Questions

- How do you build software that meets someone else's needs?
- How do you build quality attributes into software and software systems?
- How do you find, avoid, and correct defects in software and software systems?
- How do you do all of the above on a schedule and a budget?

# Class Exercise (Two minutes)

- Think about and write down three or four benefits you gain from your ‘favorite’ pieces of software: they could be mobile apps, websites or services, software that runs on a laptop, etc…,

- Think about and write down the three or four frustrations you have with software that you use.

# Software Quality Attributes

- Functionality
- Quality
  - Performance
  - Efficiency
  - Reliability
  - Usability
  - Security
  - Maintainability
  - …

- Defects in…
  - Performance
  - Efficiency
  - Reliability
  - Usability
  - Security
  - Maintainability
  - …

# Our tools for the semester…

- Managing Complexity
- Learning
- Collaboration

# Managing Complexity

- Modularity
- Cohesion
- Separation of Concerns
- Information Hiding and Abstraction
- Coupling

# Abstraction

- Break big problems into small problems
- Separate interface from implementation
- Tools you are familiar with for this:
  - Functions
  - Classes
- Now, need to think about how to break large problems down
  - Into many interrelated classes
  - Possibly multiple programs (maybe on multiple computers)
  - May communicate by things other than function call (e.g., http)

# Abstraction example: Object-Oriented Design

One major topic of this course: OO Design

- How do we split the task into (good) classes?
- What are the interfaces between classes?
- How do we make the project resilient to changes?
  - Real code changes
  - Change is hard

# Learning

- Iteration and incrementalism
- Feedback
- Empiricism
- Experimentation

# Collaboration

"The interaction of two programmers looking over a program that either one of them could have worked out is entirely different from the interaction of two programmers working on separate parts of a whole which is too great for either one to produce.

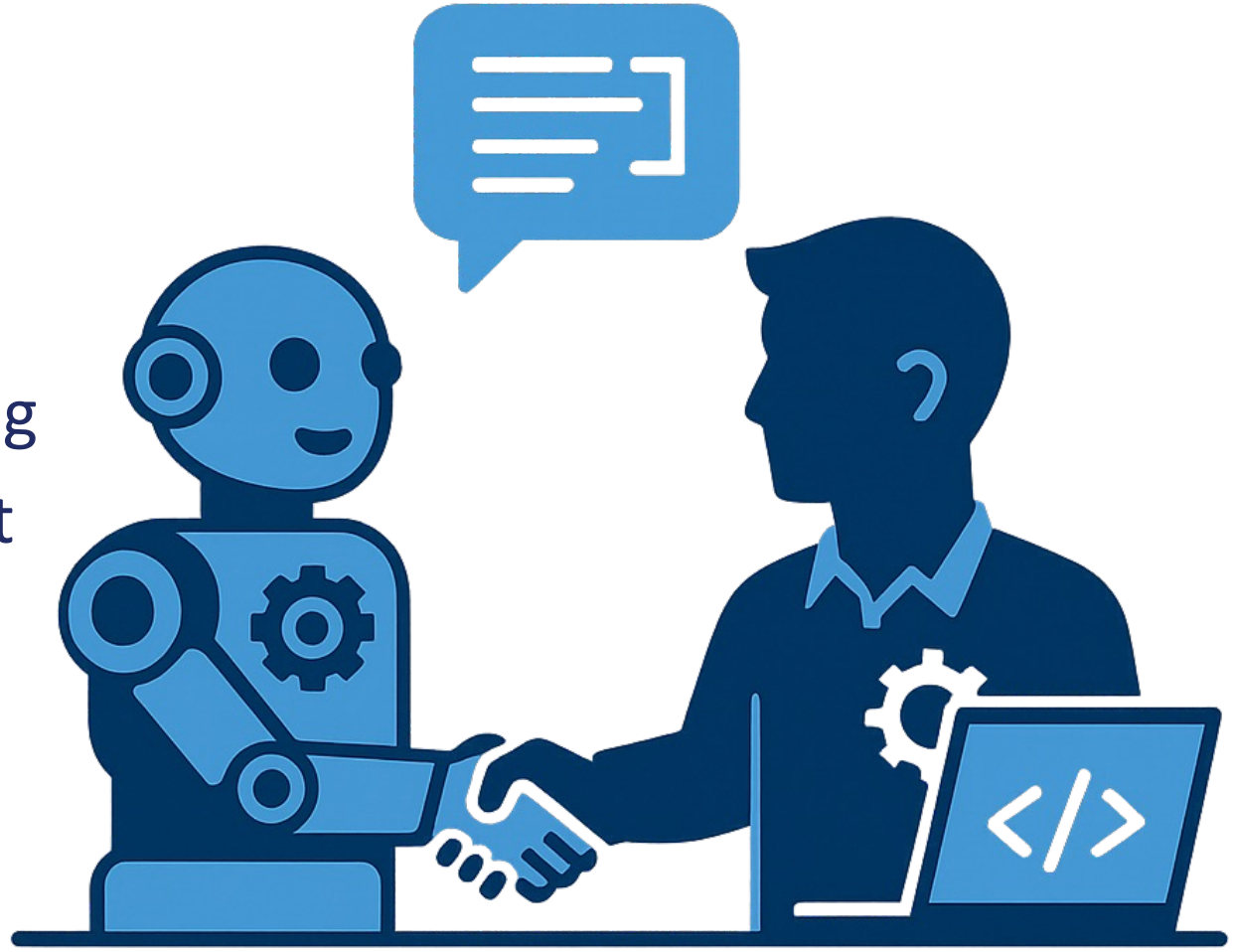
The difference lies in the way conflicting demands are resolved. In the first case, resolution of conflict is the thinking process of one person—aided perhaps by other people, but always under that one person's control. **In the second case, conflicting technical demands are translated into potential interpersonal conflicts, and a social mechanism must be formed to resolve them**.



Weinberg, Gerald. The Psychology of Computer Programming: Silver Anniversary edition

# Collaboration: Generative AI

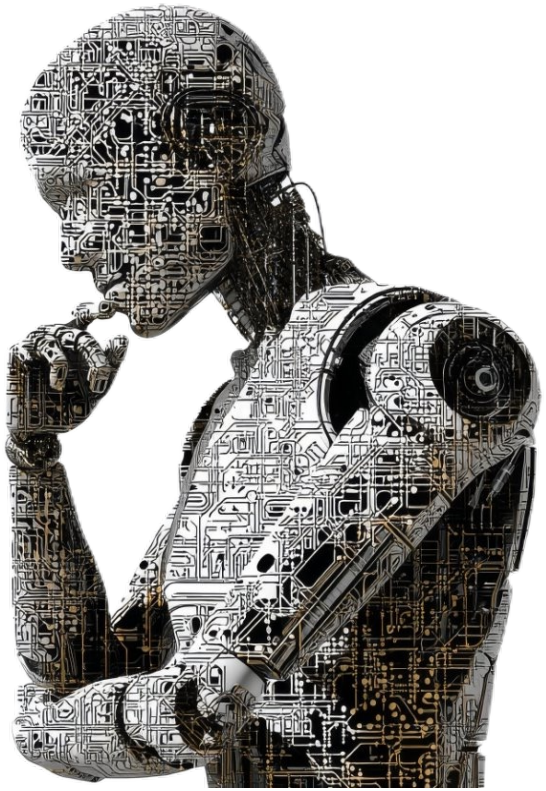
- Enhanced Communication and Knowledge Sharing
- AI-Powered Documentation
- Code Reviews and Pair Programming
- Brainstorming and Ideation Support



Example Project: URL Shortener

# AI as an Enabler (and a New Failure Mode)

- AI as an Enabler:
  - Faster prototyping
  - Lower barriers to entry
  - Productivity gains
- AI as a Risk:
  - Overconfidence
  - Hidden Assumptions
  - Loss of understanding



AI expands what teams can build and increases the corresponding responsibility.

# Course Roadmap

- Weekly lectures introducing new material
- Six weeks of weekly assignments, providing practice with the concepts and tools
- Midterm
- Team project, applying what you've learned and practiced
- Weekly lectures introducing further topics in software engineering
- No final, but will use the timeframe for Project Demonstrations (attendance is mandatory)

# Key Logistics

- Lectures: Mondays, Wednesdays, 4:40 pm EST:
- Weekly assignments are due Sundays 11:30 pm EST.
- MS Teams channels for online access to student questions and answers, TAs, instructor.
- Open to setting up Zoom ‘office hours’ in coordination with your schedule and by appointment. Will stay after class to answer questions

# Logistics: Assignments

- Weekly assignments (6) – individual effort, group discussion and support
- Team project – six-week team (5 people) software project
- Class participation – planned exercises, spontaneous comments
- Exam: midterm
- Final team presentation
- Weekly Reading

# Academic Integrity

- The problems you will face in the future will be different than the problems of the past, so you will need practice in finding your own solutions to them.
- You are unique; figuring out what works best for you is best done by you
- Therefore: Your own intellectual effort is key, both for your own benefit and your ability to contribute to a team.
- Three policies: open, individual, team

# Academic Integrity

- **Class Participation**: **open**
- Many class exercises are think-pair-share: expect you to talk to each other
- About participation, not correctness
- Class sessions are intended to be an open discussion, nothing is against the rules

# Academic Integrity

- **Weekly Assignments**: **individual/open**
- Industrial software engineering is both an individual intellectual effort and a collaborative effort
- Do your individual work first
  - Write your own code, read the paper(s), make your own comments, form your own impressions, then…
  - Bring your problems and ideas to group sessions
    - “I don’t understand <X>, can you explain it?”
    - “I liked/didn’t like <Y> about this paper.”
    - “The professor is wrong, things are really like <Z>”

# Academic Integrity

- **Team Project: team**
- Work done entirely by your team
- Discuss with teammates
  Share code with teammates
  Do not look at other team's code

- Document the source of your code as needed
- Think of this like a company: Do not expose your company to an IP infringement lawsuit!

# Academic Integrity

**Exams**: **individual -** Do not discuss at all with other students

- Can ask professor/TA clarifying questions
- May bring one page of notes
  - Front and back
  - Standard 8.5"x11"

# Assignment Schedule

- Assignments 1-6: weekly, beginning today
- Project: Begins after the weekly assignments – Financial Application
  Will also add team-based assignments
- Weekly Reading from a software engineering book of your choice